

The Knight Exchange Puzzle is NP-Hard

Henry Siegel

Abstract

Given a chessboard with holes containing an equal number of white and black knights, find a sequence of moves to swap them. This is a classic puzzle that dates back to the 1500s, and later appeared as recreational puzzles like in a video-game called *The Eleventh Hour*. We show that the decision problem that asks whether a sequence of $\leq k$ moves exists is NP-hard.

To do so, we show a reduction to the knight tour problem on chessboards with holes. This question asks whether a knight can tour every square exactly, which is the Hamiltonian Cycle problem on Knight's graphs. For square chessboards this problem can be determined in linear time, but for general chessboards with holes, it is NP-hard. It is natural to ask how the hardness of the problem changes with board restrictions. In this paper, we show that the problem remains NP-hard for connected chessboards with holes. Similarly, a closed knight's tour is a Hamiltonian Cycle, and we show that this problem is also NP-hard for connected chessboards.

The knight exchange puzzle is an instance of Pebble Motion problems for knights graphs, where two colors of pebbles are swapped. We present a short reduction from the Hamiltonian Cycle problem on bipartite graphs to this Pebble Swap Problem, showing that the Pebble Swap Problem is NP-hard on general graphs. Then we apply the reduction when restricted to knight's graphs generated by connected chessboards, showing that knights exchange puzzle is NP-hard even when restricted to connected chessboards.

Introduction

Given a graph, imagine placing pebbles on a subset of the vertices. A move consists of shifting a pebble from its current vertex to an adjacent unoccupied vertex. This characterizes the broad class of Pebble Motion Problems. These types of problems are important in distributed systems, where it is of interest to move packets over a network efficiently. The questions that have been studied include the reachability and optimality of two configurations. Variants have been well-studied when the pebbles are distinguishable or indistinguishable, or when the graphs are restricted to trees or steiner trees [6] [8].

In this paper, we will study a variant that falls between the distinguishable and indistinguishable cases. An equal number of black and white pebbles occupy some vertices with at most one pebble per vertex, and the goal is for the white pebbles to swap places with the black pebbles. We call this problem the **Pebble Swap Problem**. One subproblem that we will study is restricting the input graphs to be only knight's graphs. The Knight Exchange (Knight Swap) Puzzle is a classic puzzle dating back to 1512; A specific instance of the puzzle was known as *Guarini's Problem* [9]. The puzzle later appeared as a recreational puzzle in a video game, *The Eleventh Hour*. In this [puzzle](#), the goal is to swap the positions of the black and white knights. The knights are the pebbles on the graph generated by the chessboard. By the end of this paper, we will show that finding an optimal solution for the knight's exchange problem for connected chessboards is NP-hard. To do so, we will show a simple reduction from the Closed Knight's Tour problem, a seemingly unrelated problem.

The Knight's Tour problem asks if a knight can tour all the squares of a chessboard once. Equivalently, it is the Hamiltonian Path problem on the knight's graph. This problem ranges from linear time to NP-hard depending on the types of chessboard. Conrad [5] showed a linear time decider for square chessboards, but McGown et. al [2] showed that the problem is NP-hard for chessboards with holes. In this paper, we will confirm a conjecture posed by McGown by showing that the knight's tour problem is NP-hard for connected chessboards. One can ask the analogous question for closed knight's tour, which is a Hamiltonian Cycle, and we show that this problem is also NP-hard for the same types of chessboard.

In the next section, we show a simple reduction from the Hamiltonian Cycle problem on bipartite graphs to the Pebble Swap Problem. Then the hardness of the closed knight's tour problem on connected chessboards necessitates the hardness of the knight's exchange puzzle on the same boards, with some intermediate steps.

The Pebble Swap Problem on Bipartite Graphs

We start with a formal definition of the problem.

Definitions:

- Given a graph $G = (V, E)$, a **configuration** is a function $\mathcal{C} : V \rightarrow \{-1, 0, 1\}$ that can be thought of as the pebble assignment function where vertices mapped to -1 are occupied by a black pebble, vertices mapped to 1 are occupied by a white pebble, and vertices mapped to 0 are empty.
- A **move** is the ordered pair of configurations $(\mathcal{C}_1, \mathcal{C}_2)$ such that there exists $uv \in E$ such that $\mathcal{C}_1 \equiv \mathcal{C}_2$ on $V \setminus uv$, and $\mathcal{C}_1(v) = \mathcal{C}_2(u) = 0$ and $\mathcal{C}_1(u) = \mathcal{C}_2(v) \neq 0$.
For brevity, we will denote a move as $u \rightarrow v$, where the pebble on u moves to v .
- A **plan** is a sequence of moves of the form $\langle (\mathcal{C}_0, \mathcal{C}_1), (\mathcal{C}_1, \mathcal{C}_2), \dots, (\mathcal{C}_{k-2}, \mathcal{C}_{k-1}), (\mathcal{C}_{k-1}, \mathcal{C}_k) \rangle$. We say that the plan **transforms** $\mathcal{C}_0$ to $\mathcal{C}_k$.

Pebble Swap Problem: Given a graph G , a configuration $\mathcal{C}$ and a natural number k , is there a plan of length k that transforms $\mathcal{C}$ into $\mathcal{C}'$ so that $\mathcal{C} \equiv -\mathcal{C}'$ (the white and black pebbles have swapped places).

As shown by Akiyama [1], the Hamiltonian Cycle problem for bipartite graph is NP-Hard. We will provide a polynomial time reduction from this problem to the Pebble Swap Problem.

Theorem 0.1. *The Pebble Swap Problem on Bipartite Graphs is NP-Hard.*

Proof. Suppose the input graph is $G = (V, E)$.

We will construct the modified graph $G' = (V', E')$ as follows: Set the new vertex set V' to include all the original vertices with two extra vertices: $V' = V \cup \{x, y\}$, where $x, y \notin V$. Fix an arbitrary $v_0 \in V^+$ and add edges v_0x and v_0y . The new edge set E' contains all the original edges in E in addition to these two extra edges:

Let V^-, V^+ be the two bipartite sets of V . Place black pebbles on all the vertices in V^- and white pebbles on all the vertices in V^+ .

$$\mathcal{C}(v) = \begin{cases} 1 & \text{if } v \in V^+ \\ -1 & \text{if } v \in V^- \\ 0 & \text{if } v \in \{x, y\} \end{cases}$$

Then our transformation is $G \rightarrow (G', \mathcal{C}, n + 4)$.

Polynomial Time Reduction:

There are $O(n^2)$ edges to construct, and $O(n)$ vertices to construct in G' .

Hamiltonian Cycle $\implies$ A plan of $n+4$ moves to swap the pebbles.

Suppose $v_0v_1v_2\cdots v_{n-1}v_0$ is a Hamiltonian cycle in the input graph. (I counted up to $n-1$ because there are n vertices and we're starting at 0). Recall that v_0 is the “special” vertex that has edges v_0x and v_0y . Table 1 shows a plan of $n+4$ moves to swap the black and white pebbles on G' . For clarity, Figure 3 illustrates the plan if G is a four-cycle.

move #	move	Configuration
0	Initial	$\langle 0, 0, 1, -1, 1, \dots, -1 \rangle$
1	$(v_0 \rightarrow x)$	$\langle 1, 0, 0, -1, 1, \dots, -1 \rangle$
2 ... n	$(v_1 \rightarrow v_0) (v_2 \rightarrow v_1), \dots, (v_{n-1} \rightarrow v_{n-2})$	$\langle 1, 0, -1, 1, -1, \dots, 0 \rangle$
$n+1$	$(v_0 \rightarrow y)$	$\langle 1, -1, 0, 1, -1, \dots, 0 \rangle$
$n+2$	$(x \rightarrow v_0)$	$\langle 0, -1, 1, 1, -1, \dots, 0 \rangle$
$n+3$	$(v_0 \rightarrow v_{n-1})$	$\langle 0, -1, 0, 1, -1, \dots, 1 \rangle$
$n+4$	$(y \rightarrow v_0)$	$\langle 0, 0, -1, 1, -1, \dots, 1 \rangle$

Table 1: The configurations are $\langle \mathcal{C}(x), \mathcal{C}(y), \mathcal{C}(v_0), \dots, \mathcal{C}(v_{n-1}) \rangle$. The white and black pebbles have swapped because the starting and ending configurations are opposite.

A plan of $n+4$ moves to swap pebbles $\implies$ Hamiltonian Cycle

This is a careful counting argument. For any plan that swaps the black and white pebbles, we can analyze how the holes that start from x and y evolve after each move. We define the i th index of trajectories $\text{traj}(x)$ ($\text{traj}(y)$) as the position of the hole starting from x (y) after it has moved i times as a result of moves.

$$\text{traj}(x) := x_0x_1, \dots, x_j$$

$$\text{traj}(y) := y_0y_1, \dots, y_k$$

Note that the length of the plan is $j+k$.

Lemma 0.2.

1. $x_0, x_j, y_0, y_k \in \{x, y\}$.
2. $j, k > 0$.
3. The union of trajectories x and y visits every vertex in V : $v_0, v_1, \dots, v_{n-1}$.

Proof. Recall $V = \{v_0, v_1, \dots, v_{n-1}\} = V' \setminus \{x, y\}$ and $v_0 \in V^+$.

1. Every $v \in V$ starts with a pebble, so the holes must start and return to x and y .
2. Without loss of generality, suppose a plan didn't use y . Then the pebble at v_0 can only shift between x and v_0 . Both of these vertices aren't in V^- , which is where it needs to be after the plan.
3. For each $v \in V$, a different pebble starts and ends there. Thus, there must have been a hole there at some point, so a pebble could move there.

□

By the first two parts of lemma , trajectories x and y take the form: $\text{traj}(x) = xv_0P_xu$ and $\text{traj}(y) = yv_0P_yv$ where $u, v \in \{x, y\}$, and P_x and P_y are paths of even length that end with v_0 (they could be empty). If there was a plan of length $n+4$ that swapped the pebbles, then I claim that P_x or P_y is empty. Suppose not. Then the trajectories take the form: $\text{traj}(x) = xv_0P'_xv_0u$ and $\text{traj}(y) = yv_0P'_yv_0v$ where $P'_xv_0 = P_x$ and $P'_yv_0 = P_y$. By the third part of lemma , $P'_x \cup P'_y = \{v_1, \dots, v_{n-1}\}$, so $|P'_x| + |P'_y| \geq n-1$. The length of the plan is $j+k = |\text{traj}(x)| + |\text{traj}(y)| - 2$. Note that $|\text{traj}(x)| + |\text{traj}(y)| = 8 + |P'_x| + |P'_y|$ which includes the four total visits of v_0 , and one visit each of x, y, u and v . Thus, $|P'_x| + |P'_y| \geq n-1$ implies that the sum of the lengths of the trajectories is greater than $n+4$. This is a contradiction, so P_x or P_y is empty.

We can assume without loss of generality that P_y is empty, and P_x visits $v_1, \dots, v_{n-1}$. Each vertex appears exactly once in P_x because if a vertex occurred more than once, P_x would be longer than $n - 1$, contradicting the fact that the plan contains $n + 4$ moves. Therefore, each vertex occurs exactly once and v_{n-1} is connected to v_0 , so we have a Hamiltonian cycle. There is a small technicality when $n = 2$, but we can assume that all input graphs have at least four vertices. $\square$

Remark:

There are certainly input graphs that do not have Hamiltonian Cycles, and it is possible to swap the black and white pebbles on the transformed graph using more than $n + 4$ moves. Figure 1 shows an example.

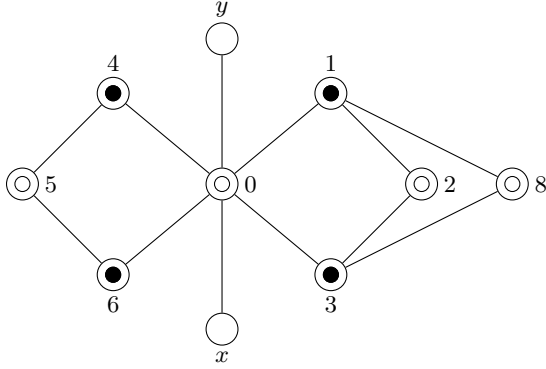


Figure 1: One can check that it is possible to swap the black and white pebbles on G' , but there is no Hamiltonian Cycle in G . Note that $\text{traj}(x)$ and $\text{traj}(y)$ both have length more than 2 for any plan that swaps the colors, so the length of the plan must be longer than 12 moves.

Jumping into Knight's Graphs

Definitions:

- A **chessboard with holes** is a rectangular lattice of squares with some subset of squares removed.
- A **connected chessboard** is a chessboard with holes with the property that a rook can travel between any two squares in some number of moves.
- The **knight's graph** of a chessboard is a graph such that the vertices are the squares of the chessboard, and the edges are the pairs of squares so that a knight can move between them in one move.
- A **knight's tour** is a Hamiltonian path on a knight's graph.
- A **closed knight's tour** is a Hamiltonian cycle on a knight's graph.

Any knight's graph is a bipartite graph, so if we can show that finding a closed knight's tour is NP-hard, then we can consider applying the reduction in the previous section to show that Pebble Swap is NP-hard on knight's graphs. This will be the direction we will take, but there is a caveat. For any connected chessboard, it is not true that we can apply the same transformation to the knight's graph by attaching two vertices to a unique vertex because the structure of the knight's graph depends on the geometry of the board. For some connected chessboards, in fact, it is impossible to add two squares a knight's distance away from a unique square, as shown in figure 10, but this is not an issue. As we will show, all instances of connected chessboards that we build will have this nice property. Alas, we begin our analysis on knight's tours and closed knight's tours.

McGown et. al [2] show that the knight's tour problem with holes is NP-complete, and they conjecture that the problem is still NP-hard on connected chessboards. We will confirm their conjecture by offering a new construction.

The team reduced from the Hamiltonian Paths decision problem in grid graphs which was shown to be NP-hard by Itai et. al [3]. We will also reduce from the Hamiltonian paths decision problem in Grid Graphs to show that knight's tours in connected chessboards is NP-hard. To show that the closed knight's tours in connected chessboards is NP hard, we will reduce the Hamiltonian cycle problem in Grid Graphs, which was shown to be NP hard by Demaine and Rudoy [4]. The same transformation in the reduction will be used to prove the hardness of both problems.

The polynomial reduction from Grid Graphs

Given an input grid graph, each vertex in the grid is replaced by a chessboard in the transformation. Let's call this mapping $a \rightarrow g(a)$, where a is the vertex in the grid graph, and $g(a)$ is its corresponding chessboard. The precise shape of each chessboard depends on the edge set of the vertex and its position in the grid graph.

Here is how we will define $a \rightarrow g(a)$:

Define the partition (A,B) on the vertex set as follows: Let (a,b) be the coordinates of a vertex in the grid graph. If $a+b$ is even, then put (a,b) in A, otherwise put (a,b) in B. First, consider v in A. Assign v a four bit string $b_1b_2b_3b_4$ denoting the edges among the four possible edges that are present. b_i is 1 if and only if the i th edge is present. The directions 1, 2, 3, and 4 are up, right, down, and left, respectively. If v is on the edge of the grid, say, direction i goes out of bounds, then assign $b_i \leftarrow 1$. This distinction for edge vertices is necessary for when we show that pebble swap is NP-hard on connected knight's graphs. There are 16 total chessboards, and the edge string of v determines the board that v is mapped to. For the complete mapping, see tables 2 and 3.

Now consider $v \in B$. Its edge string is $b_2b_1b_4b_3$. Let Q be the chessboard from this bit string, in matrix form, where 1 is a square and 0 is a hole. Rotate the elements of Q by π (equivalently reverse the rows and columns of Q). Then apply the transpose operation. Map v with the resulting board.

We will glue the chessboards together so that if a shares a vertical edge with b in the grid graph (and a is higher than b), the center of $g(a)$ is situated 9 units above $g(b)$. Perform an analogous procedure for horizontal edges. Finally, we define the total chessboard to be all the glued together pieces. Then a and b share an edge if and only if $g(a)$ and $g(b)$ share a two square bridge in the total chessboard. Hence, the grid graph is connected if and only if the total chessboard is connected. We can assume that we are working with connected grid graphs because otherwise we can check in polynomial time that the grid graph is not connected, and there is no Hamiltonian cycle and path. For a concrete example of a grid graph with the transformation, see figure 4.

Hamiltonian Cycle in grid graph $\implies$ Closed knight's tour.

Suppose that we have a Hamiltonian cycle in the grid graph. We want to show that there is a closed knight's tour. For all adjacent edges in the cycle, xy and yz , there is one square in $g(y)$ that the knight can use to go from $g(x)$ to $g(y)$, and a different square that the knight can use to go from $g(y)$ to $g(z)$. We need to show that there is a knight's tour in $g(y)$ that starts and ends at these two transition squares. Figures 5, 6, 7, and 8 enumerate all the cases for the chessboards coming from the vertices in A. The chessboards coming from the vertices in B are reversed and transposed versions of type A chessboards, so they also have a knight's tour between any two transition square. Glue all these knight's tours together and we get a closed knight's tour on the total chessboard.

Closed knight's tour $\implies$ Hamiltonian Cycle in grid graph

We adapt a lemma used in [2].

Suppose that we have a closed knight's tour. In order to show that there is a Hamiltonian Cycle in the grid graph, we need to argue two things: the closed knight's tour visits every vertex in the grid graph, and it visits each vertex exactly once. The first claim is true because the knight needs to visit every chessboard to reach all the squares. To prove the second claim, note that for every vertex v in the grid graph, the closed knight's tour cannot pass through $g(v)$ twice. Whenever the knight enters or leaves through $g(v)$, it does so by passing through the same colored square,

let us say a dark square (without loss of generality). Since a knight alternates between dark and light squares, for each pass through $g(v)$, the number of dark squares used by the knight is one more than the number of light squares. If a closed knight's tour passed through v at least twice, there would have to be at least two more dark squares than light squares in $g(v)$. Yet, $g(v)$ has exactly one more dark square, so this cannot happen. Once the knight enters $g(v)$ it must traverse every square before leaving. Therefore, there is a Hamiltonian Cycle in the grid graph.

Theorem 0.3. *The closed knight's tour problem in connected chessboards is NP-hard.*

Proof. All that is left is to prove that this reduction is polynomial time. For every v , $g(v)$ is a 9 by 9 tile, so the number of squares on the total chessboard is linear with the number of vertices on the grid graph. Checking for the position of a vertex and determining its edges can be done in polynomial time. $\square$

Theorem 0.4. *The knight's tour problem in connected chessboards is NP-hard.*

Proof. Repeat the argument used in [2] adapted for this construction. With their nomenclature, the even squares refer to the dark squares, and the odd squares refer to the light squares. The only structural difference in the constructions is if there is a Hamiltonian Path that ends in $g(v)$, and v has only one edge. Then there is a Knight's tour that visits every square in $g(v)$. Follow this knight's tour on that chessboard to complete the knight's tour on the total chessboard. $\square$

Theorem 0.5. *The pebble swap problem on knight's graphs generated by connected chessboards is NP-hard.*

Proof. Let us use the terminology that a connected chessboard is nicely extendable if we can attach two additional vertices to a unique vertex in the associated knight's graph, in such a way that the resulting chessboard is still connected. Additional squares may also be added as long as they are isolated vertices in the knight's graph, since we can keep them as holes, and no pebbles can move there. All the chessboards in the reduction are nicely extendable due to the shape of $g(v)$ for the vertices v located on the right edge of the grid graph. The details are shown in figure 9. We can conclude that the closed knight's tour problem is NP-hard for nicely extendable connected chessboards. Thus, we can apply the analogous reduction from Hamiltonian Cycles on bipartite graphs to the pebble swap problem to show that the pebble swap problem is NP-hard on knight's graphs for connected chess boards that are nicely extendable. The set of all connected chessboards is a superset of those that are nicely extendable, so the pebble swap problem is NP-hard for connected chessboards. $\square$

Corollary 0.6. *This implies that the pebble Swap Problem on (all) knight's graphs is NP-hard.*

Corollary 0.7. *All of the problems are also NP-complete on knight's graphs because they are each NP.*

Further Directions and Related Works

Now that we established the hardness results for the optimality of the knight's exchange puzzle, a natural direction to take is an algorithmic lens. Given that an instance of the puzzle is feasible, what is an algorithm that swaps the knights. For the distinguishable case on a tree with N vertices, n pebbles, and longest isthmus $k \in O(1)$, then the fastest known algorithm is $O(Nn + n^2 \log(\min\{n, k\}))$ number of moves. For general graphs, it is $O(N^2 + \frac{n^3}{N-n} + n^2 \log(\min\{n, N-n\}))$ [8]. For the indistinguishable case on a tree with N vertices, a simple algorithm with $O(N^2)$ moves exists [7]. Sufficient conditions on feasibility have also been established. For an arbitrary graph with N vertices and n pebbles, the distinguishable and indistinguishable cases are feasible if the longest isthmus in the graph is less than $N - n$. [6].

To the best of my knowledge, analogous feasibility and algorithmic results about the knight's exchange puzzle do not exist.

References

- [1] T. Akiyama, T. Nishizeki, and N. Saito. *NP-completeness of the Hamiltonian cycle problem for bipartite graphs*. *Journal of Information Processing*, 3:73–76, 1980.
- [2] K. McGown and A. Leininger. *Knight’s Tour*. Oregon State University REU Proceedings, August 15, 2002.
- [3] A. Itai, C. H. Papadimitriou, and J. L. Szwarcfiter. *Hamiltonian Paths in Grid Graphs*. *SIAM Journal on Computing*, 11(4):676–686, November 1982.
- [4] E. D. Demaine and M. Rudoy. *Hamiltonicity is Hard in Thin or Polygonal Grid Graphs, but Easy in Thin Polygonal Grid Graphs*. arXiv preprint arXiv:1706.10046, June 2017.
- [5] A. Conrad, T. Hindrichs, H. Morsy, and I. Wegener. *Solution of the knight’s Hamiltonian path problem on chessboards*. *Discrete Applied Mathematics*, 50(2):125–134, 1994.
- [6] D. M. Kornhauser, G. L. Miller, and P. G. Spirakis. *Coordinating Pebble Motion on Graphs, the Diameter of Permutation Groups, and Applications*. MIT Laboratory for Computer Science Technical Report MIT-LCS-TR-320, 1984.
- [7] S. Ardizzoni, I. Saccani, L. Consolini, M. Locatelli, and B. Nebel. An algorithm with improved complexity for pebble motion/multi-agent path finding on trees. *Journal of Artificial Intelligence Research*, 79:483–514, 2024. <https://doi.org/10.1613/jair.1.15249>
- [8] T. Nakamigawa and T. Sakuma. *On the order of the shortest solution sequences for the pebble motion problems*. CoRR, abs/2503.20550, 2025. <https://arxiv.org/abs/2503.20550> :contentReference[oaicite:0]index=0
- [9] J. J. Watkins. *Across the Board: The Mathematics of Chessboard Problems*. Princeton University Press, 2004. Chapter 1: “Introduction” (available online at Princeton University Press). <https://assets.press.princeton.edu/chapters/s7714.pdf>

Appendix

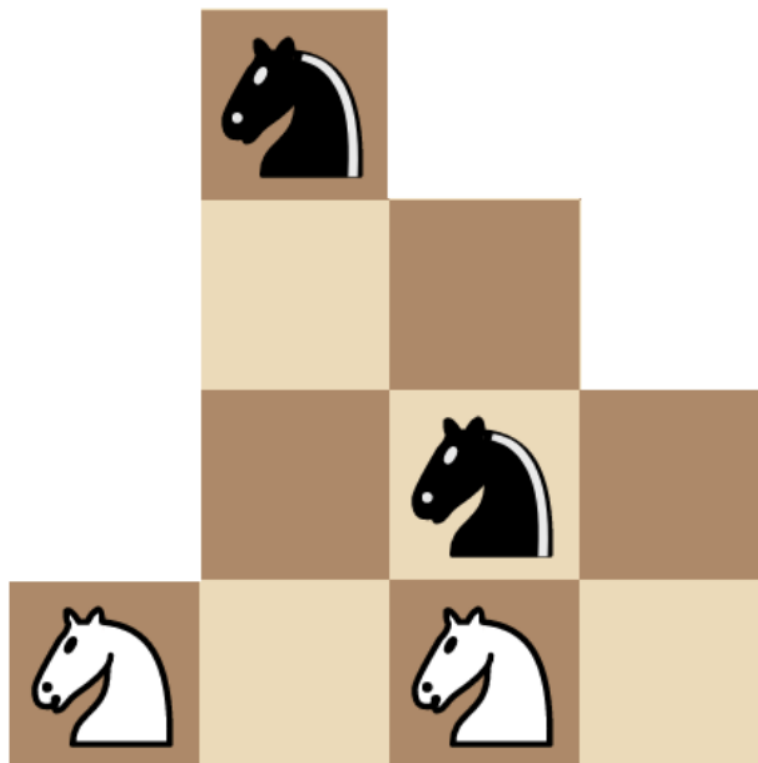
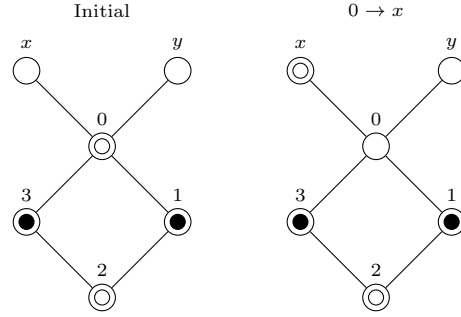


Figure 2: *The Eleventh Hour* video game puzzle: Swap the black and white knights

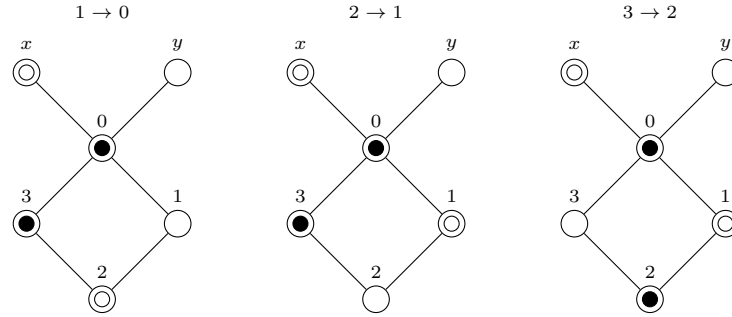
Move #

Picture

0 through 1:



2 through n:



n+1 through n+4:

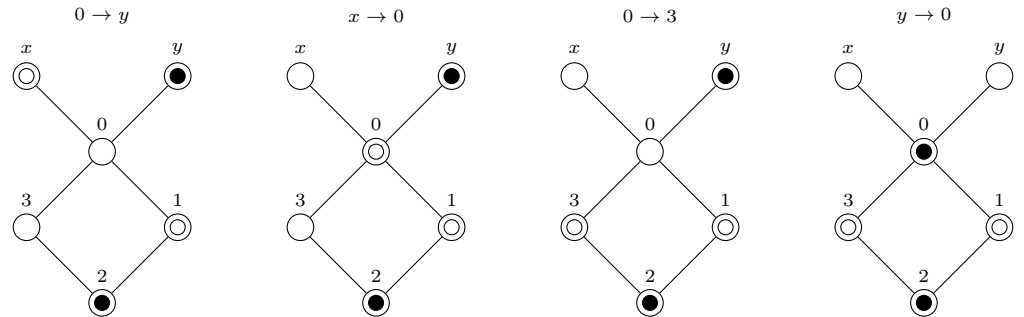


Figure 3: G is a four-cycle, and G' contains two additional vertices x and y attached to 0 as shown. In this example, $n = 4$.

Edges	Board	Edges	Board
$(0,0,0,0)$	$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$	$(1,0,0,0)$	$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$
$(0,0,0,1)$	$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$	$(1,0,0,1)$	$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$
$(0,0,1,0)$	$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$	$(1,0,1,0)$	$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$
$(0,0,1,1)$	$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$	$(1,0,1,1)$	$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$

Table 2: The mapping between edge strings and boards are shown above where the edge strings are in the order (up, right, down, left). For the edges, a 1 indicates the presence of an edge in that direction, and 0 indicates the absence of an edge. For the boards, a 1 indicates the presence of a square, and a 0 indicates the absence of a square. The $(1,1,1,1)$ board has four transition squares. To construct the other boards whenever an edge isn't present we delete the square sticking out and one adjacent square next to it. That way, the number of dark and light squares are conserved, and there is no longer a transition square for that edge. No Hamiltonian Cycle or Path will include an isolated vertex, so $(0,0,0,0)$ is mapped to the empty board. The mappings are continued in the next table.

[illegible]

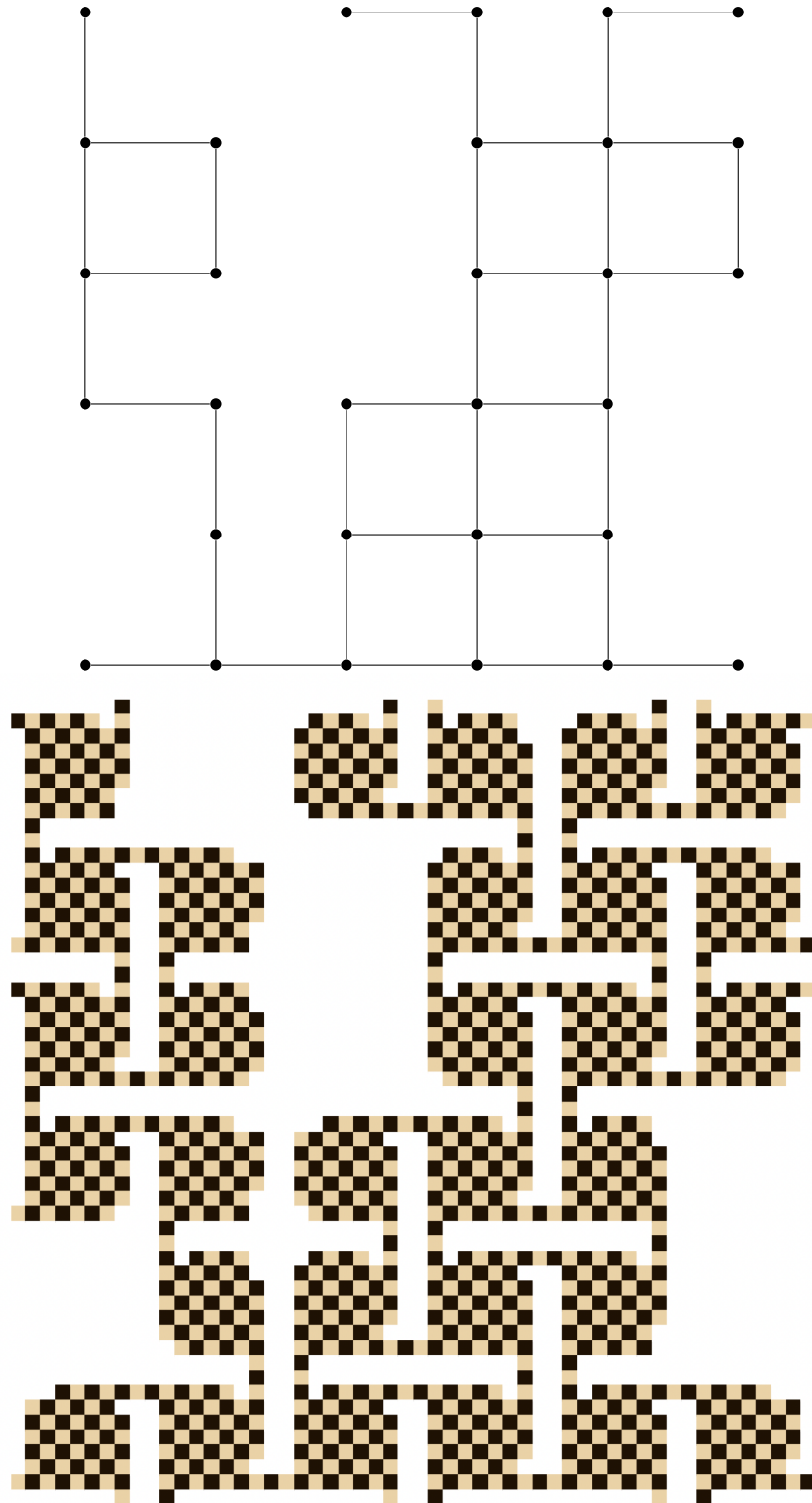


Figure 4: A grid graph and the resulting construction is shown. Each vertex in the grid graph gets mapped to a 9 by 9 chessboard with holes. There is only one way to get between any two adjacent chessboards that are connected by an edge in the grid graph, and there is no way to get between them if they are not connected. Also notice that all the transition squares are the same color, and the number of squares of this color is the majority by 1 for each chessboard.

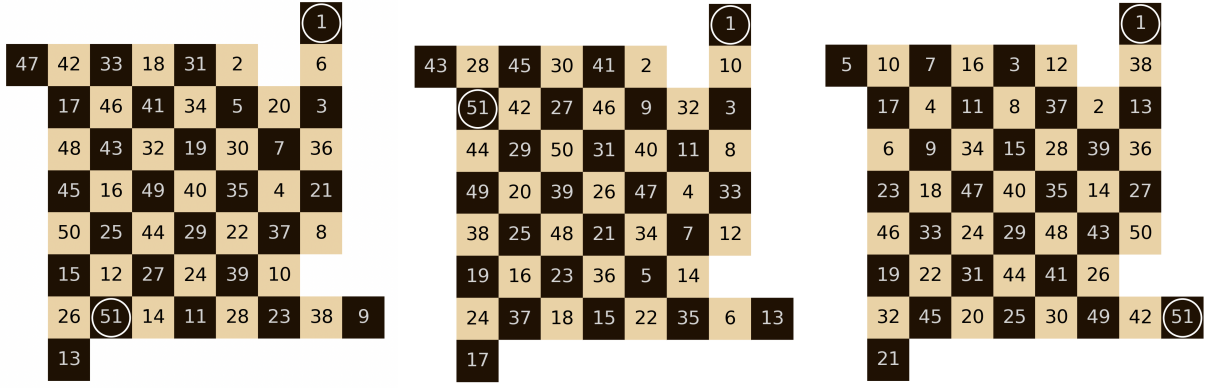


Figure 5: A board is k -connected if its vertex in the grid graph has k edges. Transition squares are entry/leaving points to other chessboards. For every pair of transition squares on the unique 4-connected board up to symmetry, there is a knight's tour. The visit times on each square is enumerated, and transition squares are circled.

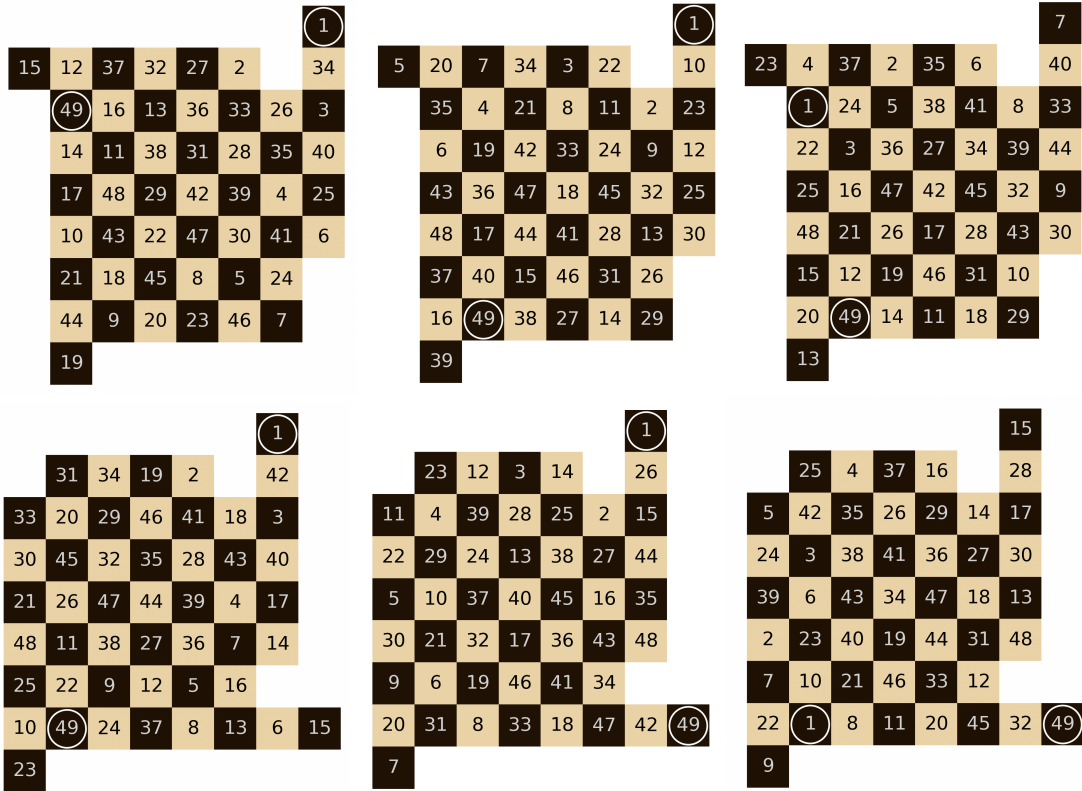


Figure 6: For every pair of transition squares on two 3-connected board up to symmetry, there is a knight's tour.

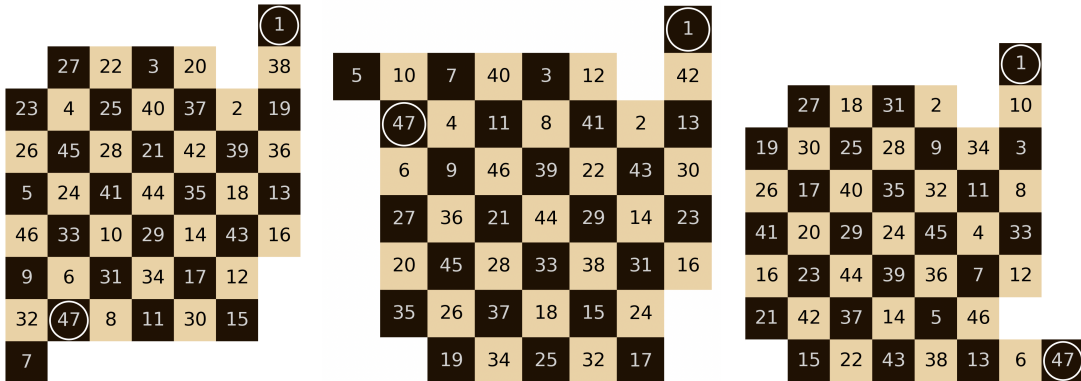


Figure 7: For every pair of transition squares on three 2-connected board up to symmetry, there is a knight's tour.

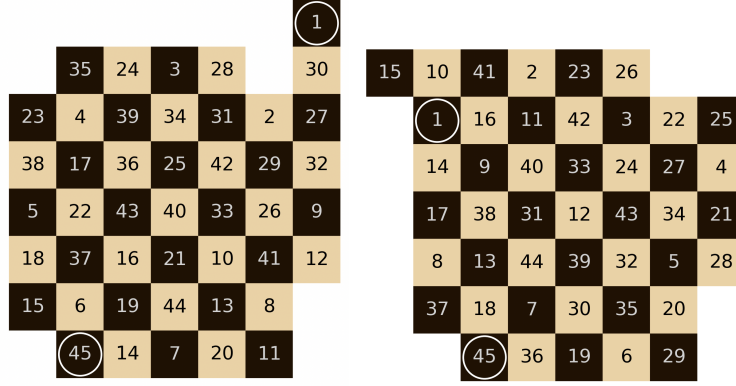


Figure 8: For both types of 1-connected boards, up to symmetry, there is a knight's tour starting on the transition square. For these boards, we only care about the starting square being a transition square because any knight tour that enters has to stay here.

This is an exhaustive list for all chessboards up to symmetry. What we showed is that for all possible pairs of adjacent edges on a Hamiltonian Cycle/Path, there is a knight's tour between the two transition squares on the chessboard that came from the vertex the edges are incident to.

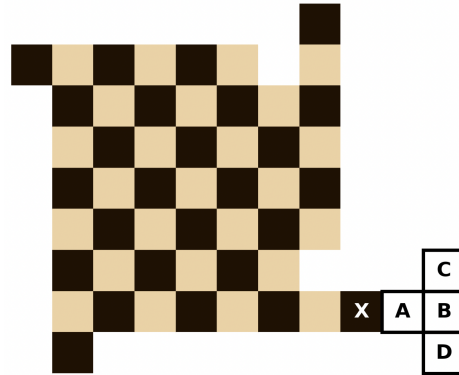


Figure 9: In our reduction, the chessboards coming from vertices on the right edge contain arm X, or a flipped version of it. When we reduce to pebble swap, we select one chessboard on the right edge and add squares A, B, C, and D. In this way, X is the only square that is a knight's distance away from C and D, while A and B are isolated vertices in the knight's graph, so their existence doesn't change the way the pebble swap reduction works: we leave A and B as holes (do not place any pebbles on them). The new chessboard is still connected. Finding vertex X can be done in polynomial time.

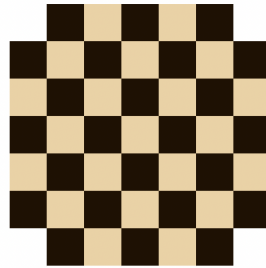


Figure 10: Shown above is an example of a chessboard that cannot be nicely extended. No matter how two additional squares are added, there are at least two squares in the original chessboard that are a knight's distance away from one of them. We don't build chessboards like this in our reduction, so this is not a problem.